# XRPL Wallet Specification

> *A modern, secure browser extension wallet for the XRP Ledger, inspired by Rabby Wallet's UX patterns and GemWallet's XRPL integration.*

---

## Table of Contents


1. Overview
2. Tech Stack
3. Architecture
4. Features
5. Security
6. Project Structure
7. Package Breakdown
8. UI/UX Design
9. Network Support
10. API & dApp Integration
11. Milestones


---

## Overview

### Vision

Build a feature-rich, security-first XRPL browser extension wallet that provides:

- ☐ **Pre-sign transaction simulation** (Rabby-style) - Show users exactly what will happen before signing
- ☐ **Risk assessment** - Warn about risky addresses, unusual transactions, or potential scams
- ☐ **Full XRPL feature support** - XRP, tokens, NFTs, DEX, escrow, checks, payment channels
- ☐ **Multi-account & multi-sign** - HD wallet derivation + multi-signature account support
- ☐ **Hardware wallet integration** - Ledger device support
- ☐ **dApp connectivity** - Custom API + WalletConnect protocol

### Inspiration

| Source | What we're taking |
|---|---|
| **Rabby Wallet** | Pre-sign simulation, risk scanning, security UX, architecture patterns |
| **GemWallet** | XRPL-specific transaction handling, API design, extension structure |

---

## Tech Stack

### Core

| Technology | Purpose |
|---|---|
| **Vue 3** | Frontend framework (Composition API) |
| **TypeScript** | Type safety across all packages |
| **Vite** | Build tool & dev server |
| **pnpm** | Package manager with workspace support |
| **Tailwind CSS** | Utility-first styling |

## XRPL Libraries

| Library | Purpose |
|---|---|
| **xrpl.js** | XRPL client, transaction building, signing |
| **ripple-keypairs** | Key generation and derivation |
| **ripple-binary-codec** | Transaction serialization |
| **bip39** | Mnemonic generation (BIP-39 standard) |

## Extension Infrastructure

| Technology | Purpose |
|---|---|
| **webextension-polyfill** | Cross-browser compatibility (Chrome/Firefox) |
| **@aspect-build/web-ext** | Extension building and packaging |

## Security

| Technology | Purpose |
| --- | --- |
| **@noble/hashes** | Cryptographic hashing (SHA-256, etc.) |
| **@noble/ciphers** | AES-GCM encryption for local storage |
| **@simplewebauthn/browser** | Passkey/WebAuthn integration |
| **@ledgerhq/hw-transport-webusb** | Ledger hardware wallet support |

### State Management & Storage

| Technology | Purpose |
| --- | --- |
| **Pinia** | Vue state management |
| **idb-keyval** | IndexedDB wrapper for encrypted storage |

### Testing & Quality

| Technology | Purpose |
| --- | --- |
| **Vitest** | Unit testing |
| **Playwright** | E2E testing |
| **ESLint** | Code linting |
| **Prettier** | Code formatting |

---

## Architecture

### Extension Components (Rabby-inspired)

```
┌──────────────────────────────────────────────────────────────┐
│                        Browser Extension                     │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  ┌──────────────┐  ┌──────────────┐  ┌────────────────────┐  │
│  │   Popup      │  │ Notification│  │      Full Tab View  │  │
│  │   (Vue)      │  │   (Vue)      │  │        (Vue)        │  │
│  └──────┬───────┘  └──────┬───────┘  └──────────┬─────────┘  │
│         │                 │                     │            │
│         └─────────────────┼─────────────────────┘            │
│                           │                                  │
│                           ▼                                  │
│  ┌──────────────────────────────────────────────────────┐   │
│  │                    Background Script                 │   │
│  │  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  │   │
│  │  │   Wallet     │  │  Provider    │  │    Keyring   │  │   │
│  │  │ Controller   │  │ Controller   │  │    Manager   │  │   │
│  │  └──────────────┘  └──────────────┘  └──────────────┘  │   │
│  └──────────────────────────────────────────────────────┘   │
│                           │                                  │
│                           ▼                                  │
│  ┌──────────────────────────────────────────────────────┐   │
│  │                   Content Script                     │   │
│  │              (Injected into web pages)               │   │
│  └──────────────────────────────────────────────────────┘   │
│                           │                                  │
│                           ▼                                  │
│  ┌──────────────────────────────────────────────────────┐   │
│  │                   Page Provider                      │   │
│  │           (window.xrpl / window.xrplWallet)          │   │
│  └──────────────────────────────────────────────────────┘   │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

## Communication Flow

```
┌──────────┐    message    ┌──────────────┐    chrome.runtime    ┌──────────┐
│   dApp   │ ────────────► │Content Script│ ──────────────────► │ Background │
│(website) │               │              │                      │   Script   │
└──────────┘               └──────────────┘                      └──────────┘
     ▲                          │                                     │
     │                          │                                     │
     └──────── response ────────┘◄──────── chrome.runtime ─────────┘
```

## Key Controllers

| Controller | Responsibility |
| --- | --- |
| **WalletController** | Account management, transaction building, network switching |
| **ProviderController** | Handle dApp requests, permission management |
| **KeyringManager** | Encrypted key storage, HD derivation, hardware wallet bridge |
| **TransactionSimulator** | Pre-sign analysis, risk assessment |

---

# Features

## Core Wallet Features

### Account Management

- [ ] Create new wallet (mnemonic generation)
- [ ] Import wallet (mnemonic, secret key, family seed)
- [ ] HD wallet with multiple accounts (BIP-44 derivation)
- [ ] Account labeling and organization
- [ ] Multi-signature account support
- [ ] Regular key management

### XRP Operations

- [ ] View XRP balance (with reserve breakdown)
- [ ] Send XRP payments
- [ ] Receive XRP (QR code generation)
- [ ] Transaction history with filtering

### Token Operations

- [ ] View issued currencies/tokens
- [ ] Set/remove trustlines
- [ ] Send/receive tokens
- [ ] Token metadata display (including xrpl.to integration)

### NFT Operations

- [ ] View NFTs (NFTokens)
- [ ] Transfer NFTs
- [ ] Burn NFTs
- [ ] Create sell/buy offers
- [ ] Accept offers

### DEX Operations

- [ ] View order book
- [ ] Create offers (buy/sell)
- [ ] Cancel offers
- [ ] Path-finding for cross-currency payments
- [ ] AMM interaction (swap, deposit, withdraw)

### Advanced XRPL Features

- [ ] Escrow (create, finish, cancel)
- [ ] Payment Channels (create, claim, close)
- [ ] Checks (create, cash, cancel)
- [ ] Tickets (create and use)
- [ ] Account settings (flags, domain, email hash)

## Security Features (Rabby-inspired)

### Pre-Sign Transaction Simulation

```
┌──────────────────────────────────────────────────────────┐
│              Transaction Simulation                      │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  📤 You're sending:                                      │
│     100 XRP → rDestination...ABC                        │
│                                                          │
│  💰 Balance after:                                       │
│     Before: 1,000 XRP → After: 899.99 XRP               │
│     (includes 0.01 XRP fee)                             │
│                                                          │
│  ⚠️ Warnings:                                            │
│     • First time sending to this address                │
│     • Large transaction (>10% of balance)               │
│                                                          │
│  ┌──────────────┐  ┌──────────────┐                     │
│  │   Reject     │  │   Confirm    │                     │
│  └──────────────┘  └──────────────┘                     │
└──────────────────────────────────────────────────────────┘
```

### Risk Scanning

- [ ] Address reputation checking
- [ ] Known scam address database
- [ ] Unusual transaction pattern detection
- [ ] Destination tag validation (for exchanges)
- [ ] Domain verification for payment pointers

### Authentication

- [ ] Password-based encryption (PBKDF2 + AES-GCM)
- [ ] Passkey/WebAuthn support (biometrics)
- [ ] Auto-lock timer (configurable)
- [ ] Session management

### Hardware Wallet

- [ ] Ledger device detection
- [ ] Sign transactions on device
- [ ] Address verification on device

## Address Book & Contacts

- ☐ ☐ Save addresses with labels
- ☐ ☐ Verified addresses (green checkmark)
- ☐ ☐ Recent addresses
- ☐ ☐ Exchange address detection (require dest tag)

---

# Project Structure

```
xrpl-wallet/
├── packages/
|   ├── extension/           # Browser extension (main app)
|   |   ├── src/
|   |   |   ├── background/  # Service worker
|   |   |   ├── content/     # Content scripts
|   |   |   ├── ui/          # Vue components
|   |   |   |   ├── popup/   # Popup view
|   |   |   |   ├── notification/  # Transaction approval
|   |   |   |   └── tab/     # Full-page view
|   |   |   ├── provider/    # Page provider (injected)
|   |   |   └── manifest/    # Extension manifests (v3)
|   |   └── vite.config.ts
|   |
|   ├── core/                # Core wallet logic (framework-agnostic)
|   |   ├── src/
|   |   |   ├── keyring/     # Key management
|   |   |   ├── transaction/ # TX building & simulation
|   |   |   ├── network/     # XRPL client wrapper
|   |   |   └── storage/     # Encrypted storage
|   |   └── package.json
|   |
|   ├── api/                 # Public API for dApps
|   |   ├── src/
|   |   |   ├── provider/    # Provider interface
|   |   |   └── types/       # API type definitions
|   |   └── package.json
|   |
|   ├── constants/           # Shared constants
|   |   ├── src/
|   |   |   ├── networks.ts  # Network definitions
|   |   |   ├── tokens.ts    # Well-known tokens
|   |   |   └── errors.ts    # Error codes
|   |   └── package.json
|   |
|   ├── types/               # Shared TypeScript types
|   |   ├── src/
|   |   |   ├── wallet.ts
|   |   |   ├── transaction.ts
|   |   |   └── api.ts
|   |   └── package.json
|   |
|   └── eslint-config/       # Shared ESLint config
|       └── package.json
|
├── pnpm-workspace.yaml
├── tsconfig.base.json
```

```
├── .eslintrc.cjs
├── .prettierrc
└── package.json
```

---

# Package Breakdown

---

## `@wallet/extension`

Main browser extension package.

**Key files:**

- ☐ `background/index.ts` - Service worker entry point
- ☐ `background/controllers/wallet.ts` - WalletController
- ☐ `background/controllers/provider.ts` - ProviderController
- ☐ `ui/popup/App.vue` - Popup main component
- ☐ `ui/notification/Approve.vue` - Transaction approval UI

## `@wallet/core`

Framework-agnostic wallet logic. Can be reused for mobile/desktop later.

**Modules:**

- ☐ `keyring/` - HD derivation, encryption, hardware wallet abstraction
- ☐ `transaction/` - Build, simulate, and sign transactions
- ☐ `network/` - XRPL WebSocket client management
- ☐ `storage/` - Encrypted IndexedDB storage

## `@wallet/api`

Published package for dApp developers.

```
// Example usage by dApps
import { XRPLWallet } from '@wallet/api';

const wallet = new XRPLWallet();

// Check if installed
if (wallet.isInstalled()) {
  // Connect
  const address = await wallet.connect();

  // Sign transaction
  const result = await wallet.signAndSubmit({
    TransactionType: 'Payment',
    Destination: 'rXXX...',
    Amount: '1000000'
  });
}
```

## @wallet/constants

Shared constants across all packages.

```
// networks.ts
export const NETWORKS = {
  mainnet: {
    name: 'Mainnet',
    url: 'wss://xrplcluster.com',
    explorer: 'https://livenet.xrpl.org'
  },
  testnet: {
    name: 'Testnet',
    url: 'wss://s.altnet.rippletest.net:51233',
    faucet: 'https://faucet.altnet.rippletest.net/accounts'
  },
  // ... more networks
};
```

## @wallet/types

Shared TypeScript interfaces.

```typescript
// wallet.ts
export interface Account {
  address: string;
  label: string;
  type: 'hd' | 'imported' | 'hardware' | 'multisig';
  derivationPath?: string;
  publicKey: string;
}

export interface WalletState {
  accounts: Account[];
  activeAccount: string | null;
  network: NetworkId;
  locked: boolean;
}
```

---

# UI/UX Design

---

## Design Principles

1. **Security First** - Every transaction shows clear simulation results
2. **Progressive Disclosure** - Simple by default, advanced features accessible
3. **Familiar Patterns** - Follow Rabby/MetaMask conventions where sensible
4. **XRPL Native** - Embrace XRPL-specific concepts (reserves, destination tags)

## Key Screens

| Screen | Purpose |
| --- | --- |
| **Welcome** | Onboarding flow (create/import wallet) |
| **Dashboard** | Account overview, balances, quick actions |
| **Send** | Send XRP/tokens with simulation |
| **Receive** | Address + QR code |
| **History** | Transaction list with details |
| **Tokens** | Token list, trustline management |
| **NFTs** | NFT gallery and management |
| **DEX** | Order book trading |
| **Settings** | Networks, security, about |
| **Approval** | Transaction approval (notification popup) |

## Component Library

Build reusable Vue components:

```
ui/components/
├── common/
|   ├── Button.vue
|   ├── Input.vue
|   ├── Modal.vue
|   ├── Card.vue
|   └── Tooltip.vue
├── wallet/
|   ├── AccountSelector.vue
|   ├── AddressDisplay.vue
|   ├── AmountInput.vue
|   ├── NetworkBadge.vue
|   └── TransactionItem.vue
├── security/
|   ├── SimulationResult.vue
|   ├── RiskBadge.vue
|   └── WarningBanner.vue
└── dapp/
    ├── DAppInfo.vue
    └── PermissionRequest.vue
```

---

# Network Support

## Built-in Networks

| Network | WebSocket URL | Type |
|---|---|---|
| **Mainnet** | `wss://xrplcluster.com` | Production |
| **Testnet** | `wss://s.altnet.rippletest.net:51233` | Test |
| **Devnet** | `wss://s.devnet.rippletest.net:51233` | Development |
| **AMM-Devnet** | `wss://amm.devnet.rippletest.net:51233` | AMM Testing |
| **AlphaNet** | `wss://alphanet.nerdnest.xyz` | Smart Contracts |

## Custom Networks

Allow users to add custom RPC endpoints:

```
interface CustomNetwork {
  id: string;
  name: string;
  url: string;
  explorer?: string;
  faucet?: string;
  networkId?: number;
}
```

---

# API & dApp Integration

## Injected Provider

The wallet injects `window.xrpl` for dApp communication:

```
interface XRPLProvider {
  // Connection
  isConnected(): boolean;
  connect(): Promise<{ address: string }>;
  disconnect(): Promise<void>;

  // Account
  getAddress(): Promise<string>;
  getNetwork(): Promise<NetworkInfo>;
  getBalance(): Promise<string>;

  // Signing
  signTransaction(tx: Transaction): Promise<SignedTransaction>;
  signAndSubmit(tx: Transaction): Promise<TxResponse>;
  signMessage(message: string): Promise<string>;

  // Events
  on(event: 'connect' | 'disconnect' | 'accountChanged' | 'networkChanged', call
  off(event: string, callback: Function): void;
}
```

## WalletConnect Support

Implement WalletConnect 2.0 protocol for standardized dApp connections:

- ☐ Support XRPL CAIP-2 chain ID format
- ☐ Handle session proposals and requests
- ☐ Persist sessions across browser restarts

### Permission System

```
interface DAppPermission {
  origin: string;
  accounts: string[];        // Allowed accounts
  methods: string[];         // Allowed methods
  grantedAt: number;
  expiresAt?: number;
}
```

---

## Milestones

---

### Phase 1: Foundation

- ☐ ☐ Project scaffolding (monorepo, tooling)
- ☐ ☐ Core package: keyring, storage, network client
- ☐ ☐ Extension infrastructure: background, content, provider
- ☐ ☐ Basic UI: create wallet, view balance, send XRP

### Phase 2: Core Features

- ☐ ☐ HD wallet with multiple accounts
- ☐ ☐ Import wallet (mnemonic, secret, family seed)
- ☐ ☐ Transaction history
- ☐ ☐ Token support (trustlines, send/receive)
- ☐ ☐ Password + passkey authentication

### Phase 3: Security & UX

- [ ] Pre-sign transaction simulation
- [ ] Risk scanning and warnings
- [ ] Address book
- [ ] Hardware wallet (Ledger) support

### Phase 4: Advanced Features

- [ ] NFT support
- [ ] DEX trading
- [ ] Multi-signature accounts
- [ ] Escrow, checks, payment channels

### Phase 5: dApp Ecosystem

- [ ] dApp API package
- [ ] WalletConnect integration
- [ ] Permission management UI
- [ ] Developer documentation

### Phase 6: Polish & Launch

- [ ] E2E testing
- [ ] Security audit
- [ ] Chrome Web Store submission
- [ ] Firefox Add-ons submission
- [ ] Landing page and documentation site

---

## Open Questions

1. **Wallet Name** - TBD

2. **Branding** - Logo, color palette, design system
3. **Analytics** - Privacy-respecting usage analytics?
4. **Monetization** - Open source only? Premium features?
5. **Smart Contracts** - Add Hooks/WASM support in future phase?

---

## References

- [ ] Rabby Wallet - Architecture and UX inspiration
- [ ] GemWallet - XRPL integration patterns
- [ ] XRPL Documentation - Ledger features reference
- [ ] xrpl.js - JavaScript/TypeScript library